

Deep Learning Based Brain Tumor Detection

Salik Ahmad

Department of Computer Science

Pakistan

GitHub: github.com/salik702 | Live App: brain-tumor-detection-salik702.streamlit.app | Portfolio: salikahmad.vercel.app

Abstract

Brain tumor detection from Magnetic Resonance Imaging (MRI) scans is a critical yet challenging task in medical image analysis. Manual diagnosis by radiologists is time-consuming and subject to human error, which motivates the development of automated deep learning solutions. This paper presents a deep learning system for binary classification of brain MRI images into two categories: No Tumor and Pituitary Tumor. The proposed system employs a Convolutional Neural Network (CNN) trained using TensorFlow and Keras, with a VGG16-based transfer learning fallback strategy to ensure robustness across deployment environments. The model achieves a training accuracy of 98% and a testing accuracy of 86%, demonstrating strong learning capability on the classification task. The system is deployed as an interactive web application using Streamlit for end-user access, and as a REST API using Flask for programmatic integration. A multi-tier fallback model-loading mechanism ensures stability across different deployment scenarios. Experimental results confirm the viability of CNN-based methods for automated MRI tumor screening.

Keywords: Brain Tumor Detection, Convolutional Neural Network, MRI Classification, VGG16, Transfer Learning, TensorFlow, Keras, Streamlit, Flask API, Medical Imaging, Deep Learning

1. Introduction

Brain tumors represent one of the most dangerous forms of cancer, with a high mortality rate if not detected early. A brain tumor is an abnormal growth of cells within the brain, and its classification — benign or malignant, and by type — plays a vital role in treatment planning. Among various tumor types, pituitary tumors are one of the most common, arising from the pituitary gland at the base of the brain.

Magnetic Resonance Imaging (MRI) is the gold standard for brain tumor diagnosis due to its high-resolution, non-invasive imaging capabilities. However, interpreting MRI scans requires expert radiological knowledge, and the manual review process is slow, inconsistent, and expensive. Deep learning, particularly Convolutional Neural Networks (CNNs), has demonstrated state-of-the-art performance in medical image recognition tasks, making it a natural candidate for automating MRI-based tumor classification.

This paper presents a complete deep learning pipeline for binary brain tumor classification — distinguishing between No Tumor and Pituitary Tumor cases — along with a fully deployed web application accessible through both a Streamlit user interface and a Flask REST API backend. The system is designed to be accessible to both medical professionals and software developers, providing real-time predictions with confidence scores.

2. Related Work

Several studies have explored deep learning for brain tumor classification from MRI scans. Havaei et al. [1] proposed a CNN-based architecture for automatic brain tumor segmentation, establishing CNNs as a strong baseline for medical imaging tasks. Pereira et al. [2] demonstrated that deep CNNs with small filters outperform handcrafted feature extraction approaches in tumor grading tasks.

Transfer learning from large image datasets has proven highly effective in medical imaging. Simonyan and Zisserman's VGG16 [3], originally trained on ImageNet, has been widely adapted for medical image tasks due to its powerful feature extraction layers. Esteva et al. [4] demonstrated that transfer learning could achieve dermatologist-level accuracy in skin cancer classification, motivating similar approaches for brain MRI.

More recent work has explored multiclass tumor detection. Cheng et al. [5] achieved over 91% accuracy on a four-class brain tumor dataset using augmentation and CNNs. Our work builds on these foundations, focusing on reliable binary classification with an emphasis on deployment readiness through web-based interfaces.

3. Methodology

3.1 Dataset

The dataset used in this study consists of brain MRI images organized into two classes: No Tumor and Pituitary Tumor. Images were collected and stored in the Brain_Tumor_Data directory within the project repository. MRI scans were sourced in standard JPEG and PNG formats, varying in resolution and orientation. The dataset covers both normal brain scans and scans containing pituitary region abnormalities, providing a balanced binary classification task.

3.2 Preprocessing Pipeline

A consistent preprocessing pipeline was applied to all input images before feeding them to the model. The pipeline consists of the following steps:

Step	Operation	Details
1	EXIF Transpose	Correct image orientation using EXIF metadata
2	RGB Conversion	Convert all images to 3-channel RGB format
3	Resize	Resize to 224 x 224 pixels (model input requirement)
4	Normalization	Scale pixel values from [0, 255] to [0.0, 1.0]
5	Batch Expansion	Add batch dimension for model.predict() compatibility

Table 1: Image Preprocessing Steps

3.3 Model Architecture

The primary model is a custom Convolutional Neural Network built with TensorFlow and Keras. The architecture follows a standard encoder pattern: stacked convolutional blocks with ReLU activations and max-pooling layers for spatial down-sampling, followed by fully connected dense layers for classification. Dropout regularization was applied to reduce overfitting.

A key design feature is the multi-tier model loading strategy, which ensures the application remains functional across varying deployment environments:

Priority	Model Format	Description
1 (Primary)	model_complete.keras	Full Keras model with architecture + weights
2 (Fallback 1)	model.json + model.h5	Separate architecture and weights files
3 (Fallback 2)	VGG16-based Builder	Reconstructed architecture using VGG16 backbone

Table 2: Model Loading Priority and Fallback Strategy

The fallback to VGG16 is particularly important for cloud environments where the primary model file may not load correctly due to library version mismatches. VGG16 weights pre-trained on ImageNet are used as the convolutional base, with custom dense layers added for binary classification output.

3.4 Model Specifications

Parameter	Value
Task	Binary MRI Image Classification
Framework	TensorFlow / Keras
Input Size	224 x 224 x 3 (RGB)
Output Classes	No Tumor (0), Pituitary Tumor (1)
Training Accuracy	98%
Testing Accuracy	86%
Loss Function	Binary Cross-Entropy
Optimizer	Adam
Fallback Architecture	VGG16 (ImageNet weights)

Table 3: Model Specifications Summary

3.5 Training Process

The model was trained on the collected MRI dataset using the Adam optimizer with binary cross-entropy loss. Image augmentation techniques — including horizontal flipping and brightness adjustments — were applied during training to improve generalization and reduce overfitting on the relatively small medical dataset. The model was evaluated on a held-out test set, achieving 86% testing accuracy, indicating good generalization with some expected gap due to the limited dataset size and class variety.

4. System Architecture

The complete system is composed of three main components: the trained deep learning model, a Streamlit-based web application for direct user interaction, and a Flask REST API for programmatic access.

4.1 Detection Pipeline

The end-to-end detection pipeline follows these sequential steps:

Step	Stage	Description
01	MRI Upload	User uploads one or more .jpg/.png MRI scan files
02	Preprocessing	EXIF correction, RGB conversion, resize to 224x224, normalization
03	Model Loading	Load model_complete.keras → fallback to model.json + model.h5 → VGG16
04	Prediction	model.predict() returns class probabilities for each input image
05	Result Display	Class label and confidence score rendered in Streamlit UI or API response

Table 4: End-to-End Detection Pipeline

4.2 Streamlit Web Interface

The primary user interface is built with Streamlit, providing a browser-based application that requires no installation from the end user. Users can access the live application at brain-tumor-detection-salik702.streamlit.app and immediately upload MRI scans via drag-and-drop. The interface displays the predicted class label and the associated confidence score, making it accessible to non-technical medical staff.

4.3 Flask REST API

A Flask-based backend API provides programmatic access to the prediction engine, enabling integration with hospital management systems, mobile applications, or other software tools. The API exposes two endpoints:

Method	Route	Description
GET	/home	Health check — returns 'Hello World' to confirm API is running
POST	/	Accepts base64-encoded MRI images, returns prediction probability scores

Table 5: Flask API Endpoints

The API accepts a JSON body with a list of base64-encoded image strings and returns a JSON response containing the raw probability scores. A score close to 0 indicates No Tumor, while a score close to 1 indicates Pituitary Tumor. Flask-CORS is enabled to support cross-origin requests from web-based client applications.

5. Technology Stack

Category	Technology	Purpose
Core Language	Python 3.x	Primary programming language
DL Framework	TensorFlow / Keras	Model training, saving, and inference

Transfer Learning	VGG16 (ImageNet)	Fallback model architecture
Web UI	Streamlit	Interactive browser-based user interface
Backend API	Flask + Flask-CORS	REST API for programmatic image prediction
Image Processing	OpenCV, Pillow	Image loading, resizing, format conversion
Data Handling	NumPy, Pandas	Array operations and data management
Deployment	Streamlit Cloud / Render	Live cloud deployment of the application
Notebook	Jupyter Notebook	Model training and experimentation

Table 6: Technology Stack

6. Results and Discussion

The proposed system was evaluated on a held-out test split of the brain MRI dataset. The model achieved 98% accuracy on the training set and 86% accuracy on the test set. The 12% gap between training and testing accuracy indicates a moderate degree of overfitting, which is common in medical imaging tasks with limited dataset sizes.

Metric	Value
Training Accuracy	98%
Testing Accuracy	86%
Input Resolution	224 x 224 pixels
Output Classes	2 (No Tumor / Pituitary Tumor)
Model Fallback Levels	3 (keras → h5/json → VGG16)
Deployment Status	Live on Streamlit Cloud

Table 7: Experimental Results Summary

The 98% training accuracy confirms that the CNN model effectively learned discriminative features from the MRI data. The testing accuracy of 86% demonstrates reasonable generalization to unseen scans. Future improvements could include dataset expansion, additional tumor classes (glioma, meningioma), and techniques such as Grad-CAM visualization for model interpretability and clinical trust.

The multi-tier fallback strategy proved especially useful during cloud deployment, where Keras version mismatches can prevent loading of .keras format models. The VGG16-based fallback ensures the application remains operational even in degraded environments, which is critical for reliability in medical settings.

7. Conclusion

This paper has presented a complete deep learning system for binary brain tumor classification from MRI scans. The system leverages a custom CNN model trained with TensorFlow and Keras, achieving 98% training accuracy and 86% test accuracy on the binary classification task of distinguishing No Tumor from Pituitary Tumor cases. The deployment architecture — combining a Streamlit web application and a Flask REST API — ensures the system is

accessible to both clinical end users and software developers.

The multi-tier model-loading strategy adds robustness that is critical for real-world deployment, where environment-specific issues can prevent standard model loading. The live deployment at brain-tumor-detection-salik702.streamlit.app demonstrates the practical viability of the system.

Future work will focus on expanding the classification to include additional tumor types (glioma, meningioma), increasing dataset diversity, applying Grad-CAM or SHAP for explainability, and exploring more advanced architectures such as EfficientNet or Vision Transformers for improved test-time accuracy.

References

- [1] Havaei, M., Davy, A., Warde-Farley, D., et al. (2017). Brain tumor segmentation with deep neural networks. *Medical Image Analysis*, 35, 18–31.
- [2] Pereira, S., Pinto, A., Alves, V., & Silva, C. A. (2016). Brain tumor segmentation using convolutional neural networks in MRI images. *IEEE Transactions on Medical Imaging*, 35(5), 1240–1251.
- [3] Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*.
- [4] Esteva, A., Kuprel, B., Novoa, R. A., et al. (2017). Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542(7639), 115–118.
- [5] Cheng, J., Huang, W., Cao, S., et al. (2015). Enhanced performance of brain tumor classification via tumor region augmentation and partition. *PLOS ONE*, 10(10).
- [6] Chollet, F. (2018). *Deep Learning with Python*. Manning Publications.
- [7] Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 265–283.